Example

height = [4, 2, 0, 3, 2, 5]

index = 0 1 2 3 4 5

Expected Answer = 9

Legend

- Left Boundary (left)
- Bottom (popped element)
- Right Boundary (right = i)
- Current Bar (i)
- Stack Element (index)
- Trapped Water

Key Idea

When we find a bar higher than the bar at the top of stack, it becomes the right boundary. Then we can calculate water with: left, bottom (popped), right.

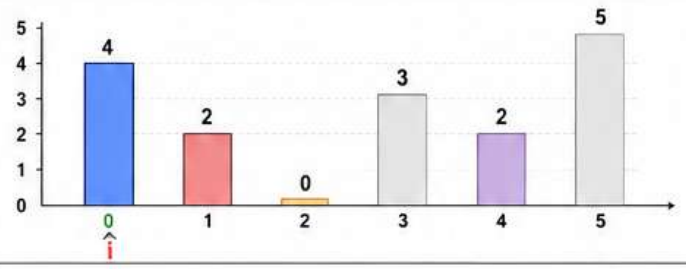
Step 0

i = 0

height[i] = 4

Stack is empty.
Push index 0.

water = 0



Stack (indices): 0 ← top

Water = 0

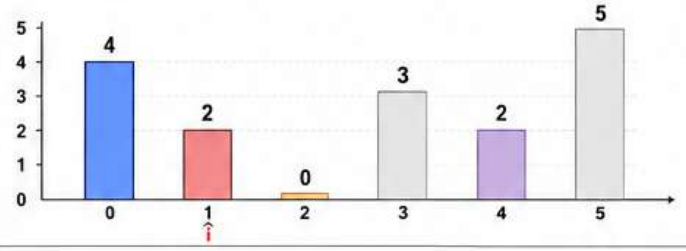
Step 1

i = 1

height[i] = 2

height[i] (2) is not greater than height[top] (4). Push index 1.

water = 0



Stack (indices): 1 ← top; 0

Water = 0

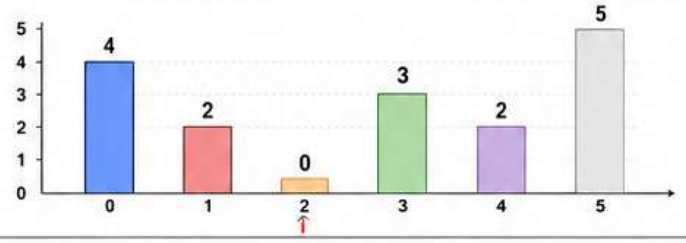
Step 2

i = 2

height[i] = 0

height[i] (0) is not greater than height[top] (2). Push index 2.

water = 0



Stack (indices): 2 ← top; 1; 0

Water = 0

Step 3

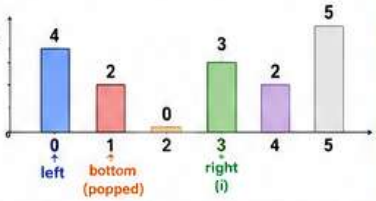
i = 3

height[i] = 3

Now height[i] (3) is greater than height[top] (0). We can calculate water.

3.1

Pop index 2 (bottom = 2).
Left = 1 (new top).
Right = 3 (current i).



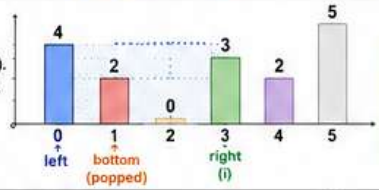
width = right - left - 1
= 3 - 1 - 1 = 1
height = min(h[left], h[right]) - h[bottom]
= min(2, 3) - 0 = 2
water = width * height
= 1 * 2 = 2

Stack (indices): 1 ← top; 0

Water = 2

3.2

Again height[i] (3) is greater than height[top] (2).
Pop index 1 (bottom = 1).
Left = 0 (new top).
Right = 3 (current i).



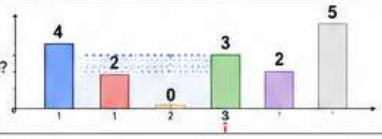
width = 3 - 0 - 1 = 2
height = min(4, 3) - 2
= 3 - 2 = 1
water = 2 * 1 = 2

Stack (indices): 0 ← top

Water = 2 + 2 = 4

3.3

Again check:
height[i] (3) > height[top] (4)?
No. Push index 3.



Stack (indices): 3 ← top; 0

Water = 4

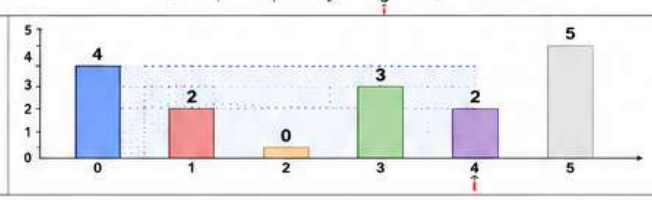
Step 4

i = 4

height[i] = 2

height[i] (2) is not greater than height[top] (3). Push index 4.

water = 4



Stack (indices): 4 ← top; 3; 0

Water = 4

Step 5

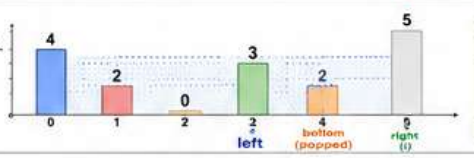
i = 5

height[i] = 5

Now height[i] (5) is greater than height[top] (2). We can calculate water.

5.1

Pop index 4 (bottom = 4).
Left = 3.
Right = 5.



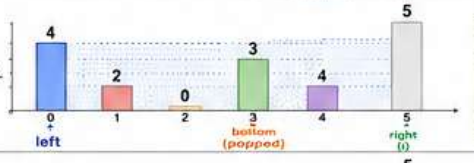
width = 5 - 3 - 1 = 1
height = min(3, 5) - 2
= 3 - 2 = 1
water = 1 * 1 = 1

Stack (indices): 3 ← top; 0

Water = 4 + 1 = 5

5.2

Again height[i] (5) > height[top] (3).
Pop index 3 (bottom = 3).
Left = 0.
Right = 5.



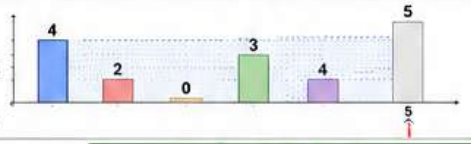
width = 5 - 0 - 1 = 4
height = min(4, 5) - 3
= 4 - 3 = 1
water = 4 * 1 = 4

Stack (indices): 0 ← top

Water = 5 + 4 = 9

5.3

Again height[i] (5) > height[top] (4).
Pop index 0 (bottom = 0).
Stack is empty now.
No left boundary.
Break and push index 5.



Stack (indices): 5 ← top

Water = 9 (Answer)

Total trapped water = 9

Input: height = [4, 2, 0, 3, 2, 5]

Index: 0 1 2 3 4 5

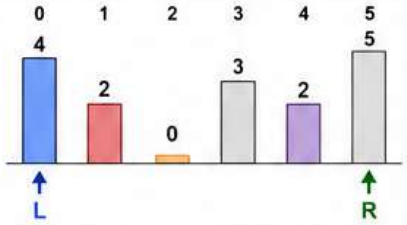
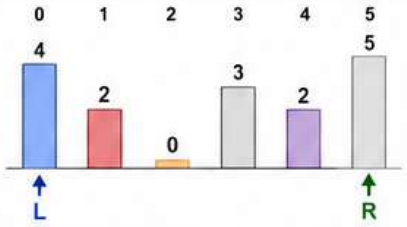
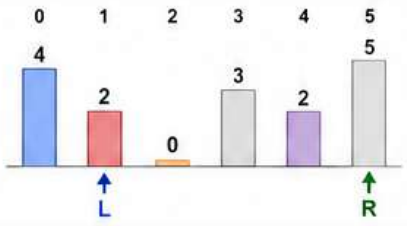
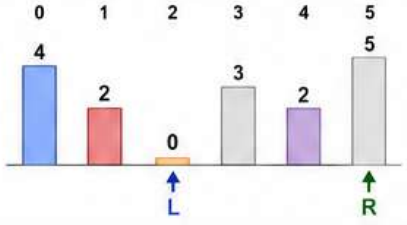
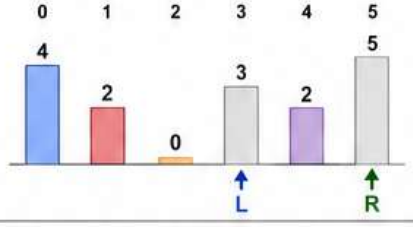
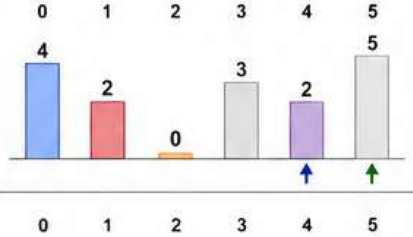
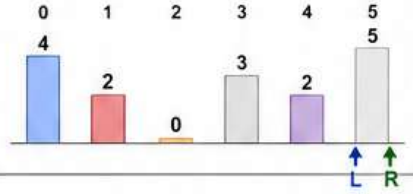
Two Pointer Approach (Optimal)

Idea

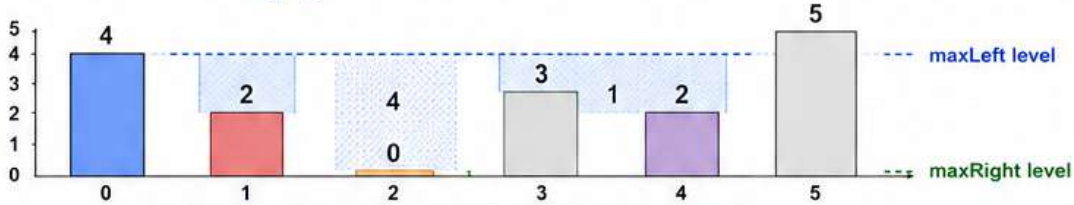
We keep two pointers, left and right.
We always move the pointer which is at the smaller height.
We maintain maxLeft and maxRight.
At each step, we calculate trapped water at that pointer's side.

Legend

- left pointer
- right pointer
- maxLeft level
- maxRight level
- water trapped

Step	Array (height)	Explanation	Pointers & Values	Water Added	Total Water
Initialization		left = 0 right = 5 maxLeft = 0 maxRight = 0 totalWater = 0	L → 0 (4) R → 5 (5) maxLeft = 0 maxRight = 0	—	0
Step 1		height[L] = 4 height[R] = 5 4 ≤ 5 → move left height[0] >= maxLeft(0) → maxLeft = 4	L → 1 R → 5 maxLeft = 4 maxRight = 0	0 (updated maxLeft)	0
Step 2		height[L] = 2 height[R] = 5 2 ≤ 5 → move left height[1] < maxLeft(4) → water = 4 - 2 = 2	L → 2 R → 5 maxLeft = 4 maxRight = 0	2	2
Step 3		height[L] = 0 height[R] = 5 0 ≤ 5 → move left height[2] < maxLeft(4) → water = 4 - 0 = 4	L → 3 R → 5 maxLeft = 4 maxRight = 0	4	6
Step 4		height[L] = 3 height[R] = 5 3 ≤ 5 → move left height[3] < maxLeft(4) → water = 4 - 3 = 1	L → 4 R → 5 maxLeft = 4 maxRight = 0	1	7
Step 5		height[L] = 2 height[R] = 5 2 ≤ 5 → move left height[4] < maxLeft(4) → water = 4 - 2 = 2	L → 5 R → 5 maxLeft = 4 maxRight = 0	2	9
Step 6 (Pointers Meet)		left == right == 5 Stop the loop. No more area to trap water.	L → 5 R → 5 maxLeft = 4 maxRight = 0	0	9

Water Trapping Visualization (Final)



Total Trapped Water = 2 + 4 + 1 + 2 = 9